<technique>
<AI/Artificial Intelligence>
인간의 지능을 모방하는 소프트웨어를 통칭하는 추상적 개념
<machine learning>
raw data에서 패턴을 학습해 일반화 하는 학습 방법
[지도 학습]
정답을 주고 정답을 구하도록 하는 지도식 설명
[비지도 학습]
정답없이 패턴을 찾아 지름이 설명
[강화학습]
보수가 가장 높아지는 방향으로 지름이 설명
<classical ML>
raw data에 직접인 feature에서 패턴을 찾아 지름이를 학습 하는 방법
<deep learning>
raw data 전체에서 패턴을 찾아 지름이를 설정하는 학습 방법
<Large Language Models>
다음 토큰의 확률 분포를 예측하기 위한 트랜스포머 설정하는 모델
<Table-Augmented Generation>
<text-to-SQL>
테이블 데이터를 대상 자연어를 DB 쿼리로 변환하는 기법
<Retrieval-Augmented Generation>
<text 검색 증강>
사전 구축된 벡터 DB에서 지식 유사한 부분을 찾아 반영하는 기법
<knowledge-based AI>
사람이 직접 규칙/지식을 정의하면, 정의된 그대로 동작
<Database Management System>
<데이터베이스를 관리하는 소프트웨어>
[Database]
파일 시스템의 중복성, 동시성 문제를 해결하기 위해 고안된 데이터 저장 방식.
[Transaction]
구분되는 단위 작업 묶음 (예: 은행 이체 = 출금 + 입금)
[Relational Database]
<데이터를 정형화하여 테이블 구조로 구축하고 관계를 기반으로 동작하는 시스템>
ACID 특성을 보장하는 DBMS 시스템을 통해 관리되는 데이터는 관계(relation) 단위로 묶여 저장되며, 테이블 간 관계는 공통 속성(attribute)을 통해 표현된다.
테이블 간 관계를 설정하여 사용된다.
<DQL>
(Structured Query Language)
relational database language
[transaction]
DB process unit
Atomicity: commit & rollback
Consistency: constraint
Isolation: race condition prevent
Durability: logging
[trigger]
정의서, 상태값/속성/이벤트 기반의 동작이 수행되는 트랜잭션
<DDL>
(Data Define Language)
<DDL-keyword>
[CREATE]: 정의 구조 생성
[CREATE Table]Type Table(Name, ...)
[ALTER]: 정의 구조 변경
[ALTER Table]Table ADD | ALTER | DROP COLUMN
[DROP]: 정의 구조 삭제
[DROP Table]Table CASCADE | RESTRICT
<Database structure>
[SCHEMA, DOMAIN, VIEW, INDEX, TABLE
[SCHEMA]
(인덱싱된 구조)(테이블, 뷰, 데이터베 그룹화) 정의
[DOMAIN]=AS type CHECK(제약조건)
(제약조건을 사용하여 사용자 정의 속성 타입 생성
[INDEX]=ON table(attribute)
(binary search 등의 탐색을 위해 사용된 정렬된 형태 생성해서는 별도 저장 구조
[VIEW]=AS SELECT – FROM –)
SELECT 문의결 조회하여 그 테이블 인상 정의됨
[TABLE]=(type attributeName, ..., CHECK(제약조건), ...)
(attribute, type, constraint 정의 테이블 생성)
attributeNot(column) 기준에 대한 생성)
– key: hash를 구분짓는 고유한 값들로 이루어진 속성들
Oracle: CHTYPE
(사용자 정의 데이터 타입 생성 시 CREATE TYPE 키워드 사용
(CREATE TYPE 이름 AS OBJECT (...)
정의 된 데이터 타입을 테이블 컬럼에 사용 가능
<DML>
(Data Manipulation Language)
<DML-keyword>
[SELECT]: 테이블 내 데이터 검색(조회)
[INSERT]: 테이블 내 데이터 삽입
[INSERT INTO]=INSERT(table, (VALUES(VALUE), ...)
[UPDATE]: 테이블 내 특정 속성 변경
[UPDATE]=UPDATE table SET 속성명=값[,속성명=값] [WHERE 조건];
[DELETE]: 테이블 내 레코드 삭제
[DELETE] FROM=FROM 테이블 WHERE 조건;
<SELECT>
SELECT [DISTINCT] (table.)column [AS rename], ([group function,
FROM table [AS rename],
* 조건절
[WHERE NOT] condition
[WHERE condition AND | OR condition]
[WHERE attribute IS NULL]
[WHERE attribute [NOT] IN (subQuery)]
[WHERE attribute BETWEEN condition AND condition]
[WHERE condition IS [NOT] NULL]
* 순서 정렬
[ORDER BY attribute ASC(디폴트)|(DESC(내림차순),)]
* 그룹핑 집계 정렬
[GROUP BY attribute, ..HAVING condition]
* 부분 집합(서 뷰만 추가)
[UNION(합집합)|INTERSECT(교집합)|MINUS(차집합)]
INTERSECT(교집합) (EXCEPT(차집합)
SELECT attribute, . FROM table,)
* 다수 집합(서 뷰), 추가)
[INTER JOIN(교집합)
LEFT OUTER JOIN(전부+오른쪽교집합)]
RIGHT OUTER JOIN(전부교집합+오른전전체)]
CROSS JOIN(카테시안)
table ON condition,)
<time query function>
[COUNT(attribute)] 개수
[SUM(attribute)] 합
[AVG(attribute)] 평균
[MIN(attribute)] 최소값
[MAX(attribute)] 최대값
[STDDEV(attribute)] 표준편차
[VARIANCE(attribute)] 분산
[ROLLUP(attribute,)] 소계 추가
[CUBE(attribute,)] 총계+소계
<like-wildcard character>
[%] 모든 문자
[ ] 단일 문자
<DCL>
(Data Control Language)
<DCL-keyword>
[COMMIT]: 영향이 수행 값과 저장
[ROLLBACK]: 영향이 수행 이전으로 복귀
[SAVEPOINT]: ROLLBACK 위치 지정
[GRANT]: 영향이 권한 부여
[GRANT 권한명 ON 테이블 TO 대상 [WITH GRANT OPTION]
[REVOKE]: 영향에 권한 회수
[REVOKE [GRANT OPTION FOR] 권한명 ON 테이블 FROM 대상
<planning>
[Product Requirements Document]
– 초기 요구사항 및 기능 정의
[Work Breakdown Structure]
– B-400시간 단위로 작업 분할
<design>
<UI/UX design>
–구상 스케, 전체 흐름 정리
[prototype]
- wire frame -> mockup -> prototype
<software design>
[flow chart]
– 소프트웨어 내 개별적 작업 흐름 정리
[Entity-Relationship Diagram]
<hardware design>
[system architecture]
– 하드웨어, 네트워크 설정 전체 구조 정리
<software>
<API>
(Application Programming Interface)
software-software interface
<software>
finally integrated component
[application]: software for specific tasks
[middleware]: processes used indirectly through client
[code]: opcode(logic) + operand(value, address)
[data]: value + type
[program]: compilation unit code
[process]: program in memory
[file]: code in drive
[library]: reusable unit code
[module]: functional unit code
[component]: service unit code
[package]: group of unit codes
<Software Development Life Cycle>
[Integrated Development Environment]
development toolset
<implementation>
[framework]
[library]
Implementation
toolset
<Inversion of Control>
hot calls code
<operation>
사람과 처리를 연결하
<control statements>
[conditional statement]
<if>
<switch/case>
[iteration statement]
<for>
<while>
<do>
[branch statement]
<return: 함수를 종료하고 호출자(main)로 이동>
<break: 현재 반복문 빠져나옴>
<continue: 현재 반복문 중단하고 다음 반복 시작>
<try-catch>
[try] 상황을 감지
[catch] 뒤에서 오류 발생 시 실행할 코드
<Interpreter>
<Compiler>
<Hybrid>
<JIT>
<operator>
[increment & decrement]
++i 증가 – i 감소
[arithmetic]
*곱셈/나눗셈/나머지 %(MOD)
+덧셈/가 -뺄셈
[relational]
<(미만) >(이상) (이상) (이하) !=(다름) ==(같음)
[logical]
!(NOT) &&(AND) ||(OR)
[bit]
&(and) ^(xor) |(or) ~(not)
<<(left shift) >>(right shift)
(floating point): 부동하지 않아서)
[array]
T [true | false]
[assignment]
=(대입) ...
<연산자 우선순위>
[연산 > 연결 > 비교연산 > 조건 > 대입]
<algorithm>
<문제해결 로직
[logic]
<최선 공식
<search algorithm>
[linear search]
<처 첫부터 값과의 유사여부 검사>
[binary search]
<정렬된 상태에서 중간값과 비교하여 절반씩 줄여 값과 유사값과 빠르게 탐색 할 때 사용>
<graph algorithm>
[Depth First Search]
–전위 순회
–중위 순회
–후위 순회
[Breadth First Search]
<루트부터 레벨별로 노드를 모두 탐색하여 최단 경로 찾기>
[Minimum Spanning Tree]
[shortest path]
<string search algorithm>
[brute force]
<단어나 문 단위 일치여부 판별 알고리즘>
[Rouyer Moore]
<문자열 내 패턴을 빠르게 찾기 위해 사용 되는 알고리즘>
<sorting algorithm>
[insertion sort] (O(N))
<정렬되지 않은 요소들을 하나씩 삽입>
[selection sort] (O(N))
<전체에서 최소값을 찾아 첫 번째 요소 자리 교환 값이 있어야 선택 비교>
[bubble sort] (O(N))
<인접 인덱 서로 비교 값이 있을 때에만 인접한 자리 값들의 교체하여 정렬>
[quick sort] (O(NlogN)~O(N))
<피벗 값 > 작은 값(pivot) < 큰 값>로 분할 반복
[heap sort] (O(NlogN))
<정렬되지 않은 요소들을 모두 힙에 넣고 큰 값이 위로 노드가 되도록 위치 교환 반복>
[merge sort] (O(NlogN))
<작은 요소가 있 정렬되어 있는 값들을 병합하여 하나로 재귀함수 정렬>
<recursive algorithm>
<자신 함수 호출하여 문제를 나눠 처리 하는 스타일 대한 접근 방법>
<재귀 깊이는 제한되어 있어 무한 호출 시 스택 오버플로우 발생>
<processing systems>
<process-copy>
[eager copy]: copy to access point
[write copy-on-write]: copy at modified point in time while referencing
<task-programing>
[multi-process]: 개개 프로세스를 동시에 실행하는 것을 말하다. 시분할 방식을 통해 하나 이상의 프로세스가 동시에 실행되는 것처럼 보이는 것이다. 운영체제는 프로세스 생성과 종료, 자원 할당을 관리하며, 각 프로세스는 독립된 메모리 공간을 유지하게 된다. 이 방식은 안정성과 보안성이 높지만, 프로세스 간 통신(IPC)이 필요하고 자원 소모가 크다는 특징이 있다.
[multi-thread]: 하나의 프로세스 안에서 여러 개의 스레드를 동시에 실행하는 것을 말한다. 스레드는 프로세스 내에서 공유되는 메모리를 사용하기 때문에 프로세스 간 통신이 빠르고 자원 소모가 적다는 장점이 있다. 대신 한 스레드가 다른 스레드의 메모리를 침범하거나 데이터를 동시에 수정하는 경우 데이터 불일치 문제가 발생할 수 있으며, 이를 방지하기 위해 동기화(synchronization) 기법이 필요하다.
[multi-core]: 하나 이상의 프로세스가 여러 개의 코어(core)를 동시에 사용하는 것을 말한다. 각 코어는 독립적인 연산 장치로 작동하며, 프로세스 또는 스레드 단위로 병렬 처리가 가능하다. 이 방식은 멀티태스킹 성능을 크게 향상시키지만, 프로세스 간 통신(IPC)이 여전히 필요하며, 공유 메모리 접근 시 동기화 문제가 발생할 수 있다.
<Process Model>
[time-sharing: preemptive algorithm]
제한시간 내에 종료하지 못한 작업은 뒤로부터 2 다음 작업으로 넘어가는 방식
<processing Method>
[non-sharing: cooperative algorithm]
<Single-stream task processing: cooperative algorithm>
스케줄 작업을 분할해서 다음 작업으로 넘어가는 방식
<task execution model>
[synchronous]
<동기화된 실행 모델>
개별 작업이 동시에 실행 가능하나 순차적으로 실행되며 다음 작업은 이전 작업이 완료되기 전까지 시작되지 않는다.
[asynchronous]
<비동기 실행 모델>
개별 작업이 동시에 실행되며 이전 작업의 완료를 기다리지 않고 다음 작업을 시작할 수 있다.
<non-blocking asynchronous model>
[task queue]
<작업대기 중인 목록
[event loop]
<이벤트 발생 감지 및 콜백 실행 담당>
[event queue]
<이벤트 발생을 기다 중인 콜백 가지>
<동기화(synchronization)>
멀티 프로세스/멀티 스레드 환경에서 공유 자원을 안전하게 접근하기 위한 메커니즘.
<동시성 제어(concurrency control)>
여러 사용자나 프로세스가 동시에 데이터를 읽고 수정하는 상황에서 데이터 일관성을 유지하기 위한 기법.
<동시성 문제>
<교착 상태(deadlock)>
서로 다른 공유 자원을 점유한 채 상대방이 점유한 자원을 기다리는 상태로 모든 작업이 멈추는 현상.
<기아(starvation)>
특정 작업이 자원을 영원히 얻지 못하고 계속 대기하는 상태로 시스템 전체는 정상이지만 해당 작업만 진행되지 않는 현상.
<경쟁 조건(race condition)>
공유 자원에 대한 접근 순서가 결과에 영향을 미치는 상태.
<임계 구역(critical section)>
공유 자원이, 공유 파일, 전역 변수, 하드웨어, 소켓.
<Operating System>
<memory system>
memory layout
[physical address]: absolute address
[logical/virtual address]: relative address
<memory trade-off>
[virtual memory]: use drive as memory (swap)
<thrashing>
페이지 교체만 CPU 점유율만 높아서 CPU가 다른 일을 못하는 현상
<memory partition>
[paging] 고정 분할 방식
고정된 크기(page)로 메모리를 나누어 관리하는 방식
<process memory layout>
[kernel area] 커널(operating sys.)
[code area] opcode + operand address
[data area] static allocation, global lifetime
[heap area]: dynamic allocation, global lifetime
[stack area]: function frame, local lifetime
<segmentation>
프로세스 단위로 메모리를 할당하는 방식
<memory>
<memory-management>
<typing/type checking>
[static typing]: compile-time typing
[dynamic typing]: run-time typing
<memory allocation>
[static allocation]: compile-time allocation
[dynamic allocation]: run-time allocation
<lifetime>
[local lifetime]: lifetime in call stack
[global lifetime]: lifetime in run-time
<scope>
[static scope (lexical)]
<block scope] 변수는 선언된 블록을 그 자식 블록에서만 유효하다.
[function scope] 변수는 선언된 함수의 그 자식 함수에서만 유효하다.
[dynamic scope]
호출되어 스택에 남아 있는 동안 유효하다.
<file system>
drive layout
[block]
–smallest file data unit
[file]
–data
–file extension
[directory /folder]
–file address table
<drive partition method>
[FAT(File Allocation Table) file system]
store block addresses in linked list structure
[ntfs file system]
store block data and addresses in tree structure
<I/O system>
<drivers>
software-hardware interface
<CPU>
<CPU instruction cycle>
[instruction fetch]
[opcode decoding]
[operand fetch]
[execution]
[instruction counter]
[mode state]
<ALU>
–calculation
<control unit>
–decoding
<register>
[Program Counter]: next opcode address
[Instruction Register]: opcode
[Memory Buffer Register]: +also operand
[Memory Address Register]: address operand
[general purpose register]: operand
[flag register]: additional result state
[stack pointer]: topmost stack address
[base register]: base address
<CPU access>
[register] 0~1.5ns
[cache] 3.5~30ns
[memory] dozens ns
[drive] hundreds ns
<hardware>
<memory>
<Random Access Memory>
| 빠른 접근을 위한
| 임시 저장 공간
<Read Only Memory>
| 정해진 일 비드웨어 기본 동작을 위한
| 정해 진 저장 공간
<I/O>
<데이터를 주고받기 위한 장치>
<Network Interface Card>
[LAN card]
| 유선 NIC
[WiFi card]
| 무선 NIC
<connector>
hardware-hardware interface
<hardware>
<network>
[gateway]
network-network
interface
<computer>
hardware(executor) + software(instruction)
<computer>
<protocol>
통신 규약
[OSI 7계층]
프로토콜이 데이터가 흐름
[POU]Protocol Data Unit]
<데이터 단위 정의>
<application layer protocol>
[HTTP(HyperText Transfer Protocol)]
<웹 문서 요청 및 응답 규칙>
[URI]Uniform Resource Identifier] 사용.
[DNS]Domain Name System]
ip 자동 서비스
[DHCP]
<IP주소 및 게이트웨이
ip 자동 할당 및 관리 서비스>
[SSH]Secure Shell]
<보안 원격 접속 서비스>
[SMTP]
e-mail 서비스
[FTP]
<파일 전송 서비스>
<Application layer>
<응용 프로그램 간 데이터 교환 규칙>
[HTTP]
[HTTP]
[HTTP]
<transport layer protocol>
[TCP]Transmission Control Protocol]
connectivity, reliability
[P2P, multipoint ...]
[UDP]User Datagram Protocol]
non-connectivity, non-reliability
[broadcast, streaming service ...]
<4.transport layer>
POU: segment(TCP), datagram(UDP)
[port number]
process identifier
[connectivity]
<check for communication with 3-way handshake
[reliability]
ensures no data is missing
<network layer protocol>
[IP(Internet Protocol)]
[ICMP(Internet Control
Message Protocol)]
[RIP (OSPF ... BGP ...]
route selection algorithm
<3.network layer>
POU: packet
[IP address]
LAN identifier
[intermediary device]
<router: separate 2 connect [broadcast domain]>
<ethernet>
[IEEE 802.3 protocol]
–define encapsulation
[CSMA/CD protocol]
separate collision domain in wire network
<wireless network>
–IEEE 802.11
–CSMA/CA
<2.data link layer>
POU: frame
[MAC address]
node identifier
[line access]
–PDP (?!?)
–multipoint (?!?)
[intermediary device]
<switch: separate 2 connect [collision domain]>
[VLAN: separate broadcast domain through a switch]
<node>
[DTE]Data Terminal Equipment]
<통신 기능이 있는 최종 장치>
–client: requester
–server: responder
[DCE]Data Circuit-terminating Equipment]
<회선을 위한 장치 신호 중간
–intermediary device
<link>
–node-node interface
<1.physical layer>
POU: bits
[physical transmission line]
–wired/cable: cable
–wireless
[line connection]
[network topology]
–mesh: 모든 노드가 서로를 연결
–star: 중앙 노드를 통해 모든 노드가 연결
–bus: 하나 노드가 하나의 회선에 연결
–tree: 위의 star가 bus에 연결
–ring: 인접한 노드끼리만 서로를 연결
[intermediary device]
–repeater: signal amplification
–hub: connection amplification
–modem: digital ||| analog signal modulation
<techniques>
<distributed processing system>
<분산되어 네 노드들이
한 시 하나의 큰 시스템 처럼
사용 공유하는 시스템>
<client/server>
–P2P 구조의 비효율 해결이 목적인 분산 시스템
프로토콜이 아닌 프로그램 단위로 동작하는 시스템 서버 역할 프로세스가 요청 대기하고 있다가 요청이 오면 응답하는 구조. 보통 하나의 물리 서버에 여러 개의 서버 프로세스가 동시에 떠 있어 클라이언트가 어떤 서버에 접속할지 결정하는 라우팅 레이어가 필요하다. 서버 클러스터 내부에서 서로 통신하기 위해 내부 네트워크를 별도로 구성하는 것이 일반적이며 인증 없이 내부 요청을 신뢰하는 구조가 흔하다.
<Client/Server>
–하나의 요청에 대한 응답을 요청한 클라이언트에게 전달하는 구조
<client-server>
<client architecture>
[on-premise]
non-cloud
[Infrastructure as a Service]
– virtualization server, storage, network
[Platform as a Service]
–runtime, OS, middleware(WAS) + IaaS
[Software as a Service]
business logic, resource + PaaS
<virtual systems>
–multipoint 구조의 물리 제하지 분산 처리 시스템
<HTTP method>
[GET] (URL로 리소스 조회
[POST]: (body로) 새로운 리소스 생성
[PUT]: (body로) 리소스 전체 교체, 없으면 생성
[PATCH]: (body로) 리소스 부분 교체
[DELETE]: (URL로) 리소스 삭제
<request start-line>
URL로 작업할 서버에 요청
<HTTP method URL HTTP version
<URI]Uniform Resource Identifier>
[resource]: server data
[URL]Uniform Resource Locator]
–protocol://domain:port/path?query#fragment
–path/variable: RESTful 방식에 주로 사용되며
경로를 동적으로 구성
–QueryParam: body가 없는 URL에 파라미터를 넘겨 전송
[URN]Uniform Resource Name]
–사용되지 않음
<HTTP start-line>
<response start-line>
–상태에 대한 응답값을 전송
<HTTP version State-code Reason-phrase
<state code>
[Successful]
–OK(200): 모든 정상적 응답 성공
–created(201): 새 리소스를 생성하는 성공 성공
[Redirection]: 요청한 내용이 다른 URL로 이동되었음
–Moved Permanently(301): 요청 내용이 이 주소 로 영구적이 옮겨 redirection
–Found(302): 요청 내용이 임시로 이동되었음 redirection
–Not Modified(304): 캐시된 데이터를 사용하라는 지시로 redirection
[Accident error]
–BadRequest(400): 잘못 된 요청 오류 전송
–Forbidden(403): 인가 된 사람만 권한 없음.
–not found(404): 요청 리소스가 존재하지 않음
[Successive error]
–internal Server Error(500): 서버 내부 문제
<HTTP/Hypertext Transfer Protocol Version>
[HTTP/1.1]: 현재 가장 널리 사용되는 버전. 기본적으로 keep-alive 방식을 사용하여 하나의 TCP 연결에서 여러 요청을 순차적으로 처리할 수 있다. 다만 한 요청이 끝나기 전까지 다음 요청을 기다리는 head-of-line blocking 문제가 있다.
[HTTP/2]: 1.1의 문제를 해결하기 위해 도입된 버전. 하나의 연결 위에서 여러 요청을 동시에 처리하는 multiplexing 을 지원하며, 헤더 압축(HPACK)과 서버 푸시(server push) 기능을 추가했다. 실제 웹 환경에서 가장 많이 사용되는 버전이다.
[HTTP/3]: 기존 TCP 대신 UDP 위에 QUIC 프로토콜을 사용하는 버전. 연결 수립 시 핸드쉐이크 단계를 줄이고, 패킷 손실 시 다른 스트림에 영향을 주지 않는 구조를 가진다. 모바일 환경에서 특히 유리하다.
<request header>
[host]: 요청하는 서버의 도메인 주소.
[Referer]: 요청하는 URL 직전에 있었던 URL
[User-Agent]: Client가 사용 중인 OS, rendering engine, web-browser 정보
[Accept]/[Client가 선호하는: Content-Type
[Accept-Encoding]/[Client가 선호하는: Content-Encoding
[Accept-Language]/[Client가 선호하는: Content-Language
[Range]: 범위 전송 구간 지정
[Authorization]: 로그인 인증 정보
[Cookie]: 브라우저 cookie storage에 보관하여 cookie를 서버에 전달
[If-Modified-Since]: 서버에서 변경된 리소스에 대해서 날짜가 수정된 경우에만 body를 전송 (아니면 304 응답)
[If-None-Match]: etag(리소스 특정된 리소스의 지문)가 일치하는 수정된 경우에만 body를 전송 (아니면 304 응답)
<사용자 맞춤화>
<사용 목적과 정보>
<HTTP message>
<http client devices>
<web browser>
[rendering engine]
/html, css, javascript parsing and UI painting
[javascript engine]
<Client Side Rendering
(동적 데이터만 전송받아 클라이언트에서 html를 추가)
<web application>
<HTTP-based software to process resource
[backend]
<presentation layer(API)/controller): frontend와 통신
<business layer(service): 비즈니스 로직 처리 및 트랜잭션 관리
<data access layer(repository/DAO): 데이터 저장소와 통신
–GT(Gotchema): API 입출력 데이터 추상화
–Orbit(entity/model): DB 테이블 추상화
[frontend]
<View(component/page): UI 렌더링 담당
–State(component/hook): UI의 데이터 상태 관리
–API: backend와 통신
<Web Application Server>
<요청 처리기 웹서버, 정적 파일 제공>
[web server]: software to transfer resource to http
[application server]: software for execute application
<view template engine]
Server Side Rendering
(동적 데이터 포함한 HTML 생성)
<WWW(World Wide Web)>
By Tim Berners-Lee, 1989
인터넷 인프라 위에 정의되는 개념 (이 HTTP를 전달하는 웹사이트 통신 방식
<HTTP Header>
–http message metadata
(filed value) 구조
<general header>
[Cache-Control]
<max-age: 캐시된 리소스의 유효기간
<no-cache: origin server에 캐시된 리소스의 유효성 확인 요청
<no-store: 캐시 저장 자체를 금지 (민감한 데이터)>
<public: proxy server에 저장되어도 공용 데이터
<private: proxy server에 저장되면 안되는 개인 데이터
<representation header>
[Content-Type]: body 데이터 형식
[Content-Encoding]:body 압축 방식 명시
[Content-Language]: body 언어 언어 코드
[Content-Length]: body 바이트 길이
<HTTP Body>
[실제 전송할 데이터
(byte로 표현 가능한 모든 데이터)
<request body type>
[x-www-form-urlencoded]
<text form data 전송 방식,
body로 인코딩되면 query처럼 파싱
[application/form
–API 통신에서
–개발 표준화된 데이터 포맷
<response body type>
[text/html]
–html, css, js file
[text/plain]
–일반 텍스트 전송
[application/form
–API 통신에서
–개발 표준화된 데이터 포맷
[기타 등등]
<request header>
[Set-Cookie]
<server가 client에 데이터 기억는 과정은 고 전송
[client]: 받았던 cookie를 cookie storage에 저장
–name=value: 실제 저장되는 데이터
–domain: 쿠키를 제공하는 서버 도메인 주소
–path: domain의 어떤 경로
–expires/max-age: 쿠키 만료일(expires)/유효시간(max-age) 초 초과시 삭제
–secure: HTTPS 환경에서만 전송
–HttpOnly: 자바스크 쿠키 접근 차단
[Last-Modified]: 특정 데이터에 대해 마지막 수정된 시각
[ETag]: 특정 데이터에 대해 변경사항 확인할 때 사용하는 값
<HTTP method>
[GET] (URL로 리소스 조회
[POST]: (body로) 새로운 리소스 생성
[PUT]: (body로) 리소스 전체 교체, 없으면 생성
[PATCH]: (body로) 리소스 부분 교체
[DELETE]: (URL로) 리소스 삭제
<URI]Uniform Resource Identifier>
[resource]: server data
[URL]Uniform Resource Locator]
–protocol://domain:port/path?query#fragment
–path/variable: RESTful 방식에 주로 사용되며
경로를 동적으로 구성
–QueryParam: body가 없는 URL에 파라미터를 넘겨 전송
[URN]Uniform Resource Name]
–사용되지 않음
<request start-line>
URL로 작업할 서버에 요청
<HTTP method URL HTTP version
<state code>
[Successful]
–OK(200): 모든 정상적 응답 성공
–created(201): 새 리소스를 생성하는 성공 성공
[Redirection]: 요청한 내용이 다른 URL로 이동되었음
–Moved Permanently(301): 요청 내용이 이 주소 로 영구적이 옮겨 redirection
–Found(302): 요청 내용이 임시로 이동되었음 redirection
–Not Modified(304): 캐시된 데이터를 사용하라는 지시로 redirection
[Accident error]
–BadRequest(400): 잘못 된 요청 오류 전송
–Forbidden(403): 인가 된 사람만 권한 없음.
–not found(404): 요청 리소스가 존재하지 않음
[Successive error]
–internal Server Error(500): 서버 내부 문제
<HTTP/Hypertext Transfer Protocol Version>
[HTTP/1.1]: 현재 가장 널리 사용되는 버전. 기본적으로 keep-alive 방식을 사용하여 하나의 TCP 연결에서 여러 요청을 순차적으로 처리할 수 있다. 다만 한 요청이 끝나기 전까지 다음 요청을 기다리는 head-of-line blocking 문제가 있다.
[HTTP/2]: 1.1의 문제를 해결하기 위해 도입된 버전. 하나의 연결 위에서 여러 요청을 동시에 처리하는 multiplexing 을 지원하며, 헤더 압축(HPACK)과 서버 푸시(server push) 기능을 추가했다. 실제 웹 환경에서 가장 많이 사용되는 버전이다.
[HTTP/3]: 기존 TCP 대신 UDP 위에 QUIC 프로토콜을 사용하는 버전. 연결 수립 시 핸드쉐이크 단계를 줄이고, 패킷 손실 시 다른 스트림에 영향을 주지 않는 구조를 가진다. 모바일 환경에서 특히 유리하다.
<Client/Server>
–하나의 요청에 대한 응답을 요청한 클라이언트에게 전달하는 구조
<client-server>
<client architecture>
[on-premise]
non-cloud
[Infrastructure as a Service]
– virtualization server, storage, network
[Platform as a Service]
–runtime, OS, middleware(WAS) + IaaS
[Software as a Service]
business logic, resource + PaaS
<virtual systems>
–multipoint 구조의 물리 제하지 분산 처리 시스템
<distributed processing system>
<분산되어 네 노드들이
한 시 하나의 큰 시스템 처럼
사용 공유하는 시스템>
<client/server>
–하나의 요청에 대한 응답을 요청한 클라이언트에게 전달하는 구조
<staying-Authentication>
[session]
–서버에 로그 남아 있음
–클라이언트는 session ID만 보관
[token/Token]
–클라이언트가 자기 토큰을 서버에 제출
–서버는 토큰에서 사용자 인증정보를 검증하면 되므로 저장 부담
<First Authentication>
[ID password]
[physical 인증]
<network Authentication>
<network security>
[Confidentiality]
–encryption algorithm
–hosed
[Integrity]
–hash algorithm
–error control
[Availability]
–load balancing: distributed server
–failover: spare server
<encryption algorithm>
[symmetric key]
–암호화 키와 복호화 키 같음
[asymmetric key]
–암호화 키와 복호화 키 다름
<firewall>
traffic filtering
[<IPS]Intrusion Prevention System]
–monitoring + blocking
[>IDS]Intrusion Detection System]
–monitoring + warning